

AIOps Module 3 Assignment

Dhanush

September 19, 2026

1 Question 1: Single-Stage vs. Multi-Stage Docker Build

The naive Dockerfile uses `python:3.12` as its base and builds and runs correctly, its image size is 647MB. The multi-stage version uses a `python:3.12` builder stage to install dependencies, then copies just the installed packages into a `python:3.12-slim` runtime stage. It also builds and runs correctly with identical behavior, at 279MB, a **56.9% reduction**.

The multi-stage image is much smaller (56.9% less). The naive image builds everything in one single `python:3.12` image, which includes compilers and build tools needed to install packages like `numpy` and `scikit-learn`, these stay in the final image even though they are only needed once, during install. In the multi-stage version, the install step happens in a separate `builder` stage, and only the installed packages folder (`site-packages`) gets copied into a fresh, slim final image, leaving the compilers and everything else behind.

Image	Content Size
<code>naive</code>	647MB
<code>multistage</code>	279MB

Table 1: Q1: Image size comparison (56.9% reduction)

2 Question 2: Multi-Container Orchestration with Docker Compose

A single Dockerfile only builds one image, so if two containers (like the API and Redis) need to run together and talk to each other, plain Docker cannot start and connect them on its own. Compose solves this with one YAML file that defines multiple services and starts them together with one command, already networked so they can reach each other by name. Its scope stops at a single machine, though; Kubernetes works across a whole cluster, scheduling containers on different nodes, restarting them on failure, and scaling replicas automatically.

Message type	1st request (miss)	2nd request (hit)	Speedup
Spam-style	38.1 ms	0.73 ms	$\sim 52\times$
Ham-style	4.3 ms	0.81 ms	$\sim 5\times$

Table 2: Q2: Cache miss vs. cache hit latency

Table 2 shows the cache in action for both message types: cached requests were $52\times$ faster for the spam-style message (38.1 ms $\rightarrow$ 0.73 ms) and $5\times$ faster for the ham-style message (4.3 ms $\rightarrow$ 0.81 ms), confirming the cache works correctly on both classes.

3 Question 3: Kubernetes Indexed Job for Parallel Data Validation

3.1

Each pod requests 900m CPU. Each node has 2000m allocatable, some of which Kubernetes' own system pods need, so at 900m per pod, two pods use 1800m of a node's 2000m, leaving about 200m of headroom. That means two pods fit per node, so across 2 nodes `parallelism` is set to 4. `Completions` is set to 8 since there are 8 shards to validate. For the Downward API, `POD_NAME` comes from `metadata.name` and `NODE_NAME` from `spec.nodeName`, so each shard's result can be tied to the pod and node that produced it.

3.2

3.3

Results were collected using the Kubernetes API rather than a shared volume, since minikube's storage provisioner is node-local, and the pods in this Job run across multiple nodes. Reading logs through the API works regardless of which node a pod ran on, avoiding that problem entirely.

Shard	0	1	2	3	4	5	6	7	Total
Invalid rows	1	2	3	4	1	2	3	4	20

Table 3: Q3: Invalid rows per shard, collected via the Kubernetes API

3.4

No, the parallelism choice would not change. With completions=8 and parallelism=4, the job runs in 2 waves, using all 4 cores in both waves. If parallelism is raised to 6, the job still needs 2 waves, since $\lceil 8/6 \rceil = 2$ as well, but the second wave would only have 2 shards left to run, so 4 of the 6 cores would sit idle. The total time taken stays the same either way, so raising parallelism to 6 does not actually save any time, it just adds concurrency that never gets used.

Cluster	Total CPUs	Chosen parallelism
2 nodes $\times$ 2 CPUs	4	4

Table 4: Q3: Parallelism choice vs. available cluster CPU

4 Question 4: Kubernetes Deployments — Self-Healing and Rolling Updates

4.1

4.2

The **ReplicaSet controller** is responsible for self-healing. In the manifest, **replicas:** 2 sets the desired pod count, and **selector.matchLabels** tells it which pods to count. If a pod is deleted and the count drops below 2, it creates a replacement from the same **template** block, so the new pod matches the original exactly. This was confirmed by deleting one pod manually, a new pod appeared within seconds while the other kept running unaffected.

Event	Pod	Age at observation
Before deletion	spam-api-...-m4tds	97s
Before deletion	spam-api-...-zt9m7	97s
After deleting m4tds	spam-api-...-fhs5k (new)	8s
After deleting m4tds	spam-api-...-zt9m7 (untouched)	118s

Table 5: Q4: Self-healing evidence — deleted pod replaced automatically

4.3

A version string was added to **/healthz** via an **APP_VERSION** environment variable. A new image was built as **spam-api:v2**, and the Deployment was updated to use it. **kubectl rollout status** confirmed the update completed without downtime, replacing one pod at a time, and **kubectl rollout history** showed two revisions; **/healthz** confirmed **"version":"v2"** afterward. If this had been a Job instead, there is no ongoing replica count to maintain, so updating the image would mean deleting and recreating it from scratch, causing an outage instead of a seamless rollover.

Revision	Image	/healthz version
1	spam-api:v1	v1
2	spam-api:v2	v2

Table 6: Q4: Rolling update revisions (`kubectl rollout history`)

4.4

A Deployment is built for long-running, always-available serving workloads, it actively maintains a fixed number of replicas forever, replacing pods that crash or get deleted, and rolls out image updates gradually with no downtime, which fits an API that must always be reachable. A Job like the shard-validation workload in Question 3, is built for run-to-completion batch work, once all completions succeed, there is no ongoing state to maintain, so self-healing and rolling updates are not relevant concepts for it. In short, a Deployment fits workloads that should run forever and stay updated, while a Job fits workloads that should run once, finish, and stop.